



Міністерство освіти і науки України
Національний технічний університет України
“Київський політехнічний інститут імені Ігоря Сікорського”
Факультет інформатики та обчислювальної техніки
Кафедра інформаційних систем та технологій

Лабораторна робота №5

із дисципліни «Технології розробки програмного забезпечення»

**Тема: «ШАБЛОНИ «ADAPTER», «BUILDER», «COMMAND», «CHAIN OF
RESPONSIBILITY», «PROTOTYPE»»**

Виконав:
Студент групи ІА-24
Зелінський І.О.

Перевірив:
Мякий М. Ю.

Зміст

Тема.....	3
Мета.....	3
Завдання.....	3
Теоретичні відомості.....	3
Обрана тема.....	5
Інтерфейс SongAdapter	6
Клас FullSongToSongAdapter.....	6
Використання.....	6
Тестування.....	6
Висновок.....	7

Тема: ШАБЛОНИ «ADAPTER», «BUILDER», «COMMAND», «CHAIN OF RESPONSIBILITY», «PROTOTYPE»

Мета: Навчитися реалізовувати шаблони «ADAPTER», «BUILDER», «COMMAND», «CHAIN OF RESPONSIBILITY», «PROTOTYPE»

Завдання:

1. Реалізувати не менше 3-х класів відповідно до обраної теми.
2. Реалізувати один з розглянутих шаблонів за обраною темою.

Теоретичні відомості

Шаблони проєктування

Шаблон проєктування — це типовий підхід до вирішення поширених проблем, які виникають під час створення архітектури програмного забезпечення. Вони не є готовими бібліотеками чи функціями, які можна просто скопіювати в проєкт. Замість цього шаблон описує загальний принцип, який потребує адаптації до конкретних умов програми.

Класифікація шаблонів

Шаблони можна поділити на три основні групи:

1. **Породжуючі:** допомагають створювати об'єкти, зменшуючи кількість залежностей у програмі.
2. **Структурні:** визначають, як об'єкти можуть взаємодіяти між собою.
3. **Поведінкові:** оптимізують обмін даними та комунікацію між об'єктами.

Adapter

Шаблон "adapter" (адаптер) використовується для адаптації інтерфейсу одного об'єкту до іншого. Наприклад, існує декілька бібліотек для роботи з принтерами, проте кожна має різний інтерфейс (хоча однакові можливості і призначення). Має сенс розробити уніфікований інтерфейс (сканування, асинхронне сканування, двостороннє сканування, потокове сканування і тому подібне), і реалізувати відповідні адаптери для приведення бібліотек до уніфікованого інтерфейсу. Це дозволить в програмі звертатися до загального інтерфейсу, а не приводити різні сценарії роботи залежно від способу реалізації бібліотеки. Адаптери також називаються "wrappers" (обгортками).

Builder

Шаблон "builder" (будівельник) використовується для відділення процесу створення об'єкту від його представлення. Це доречно у випадках, коли об'єкт має складний процес створення (наприклад, Web- сторінка як елемент повної відповіді web-сервера) або коли об'єкт повинен мати декілька різних форм створення (наприклад, при конвертації тексту з формату у формат).

Command

Об'єкт команда сама по собі не виконує ніяких фактичних дій окрім перенаправлення запиту одержувачеві (тобто команди все ж виконуються одержувачем), однак ці об'єкти можуть зберігати дані для підтримки додаткових функцій відміни, логування і інш. Наприклад, команда вставки символу може запам'ятовувати символ, і при виклику відміни викликати відповідну функцію витирання символу. Можна також визначити параметр "застосовності" команди (наприклад, на картинці писати не можна) - і використати цей атрибут для засвічування піктограми в меню.

Такий підхід до команд дозволяє побудувати дуже гнучку систему команд, що настроюється. У більшості додатків це буде зайвим (використовується спрощений варіант), проте життєво важливий в додатках з великою кількістю команд (редактори).

Prototype

Шаблон "prototype" (прототип) використовується для створення об'єктів за "шаблоном" (чи "кресленню", "ескізу") шляхом копіювання шаблонного об'єкту. Для цього визначається метод "клонувати" в об'єктах цього класу.

Цей шаблон зручно використати, коли заздалегідь відомо як виглядатиме кінцевий об'єкт (мінімізується кількість змін до об'єкту шляхом створення шаблону), а також для видалення необхідності створення об'єкту - створення відбувається за рахунок клонування, і зухвалій програмі абсолютно немає необхідності знати, як створювати об'єкт.

Обрана тема: Online radio station (iterator, adapter, factory method, facade, visitor, client-server)

Додаток повинен служити сервером для радіостанції з можливістю мовлення на радіостанцію (64, 92, 128, 196, 224 kb/s) в потоковому режимі; вести облік підключених користувачів і статистику відвідувань і прослуховувань; налаштувати папки з піснями і можливість вести списки програвання або playlists (не відтворювати всі пісні).

Хід роботи

Під час розробки програми я реалізував шаблон "Адаптер", оскільки може виникнути ситуація де пісня має список виконавців, замість їх Id, а щоб зберігати структуру набагато краще все зберігати через Id, тому ми адаптуємо до базового класу Song, у якому і є список List<String> artistIds;

Інтерфейс SongAdapter:

```
package com.OnlineRadio.OnlineRadioStation.adapters;

import com.OnlineRadio.OnlineRadioStation.models.Song;

public interface SongAdapter {
    Song adapt();
}
```

Рисунок 1. Інтерфейс SongAdapter

Клас FullSongToSongAdapter, що адаптує клас FullSong, що містить список артистів до Song, де зберігаються їх Id.

```
package com.OnlineRadio.OnlineRadioStation.adapters;

import com.OnlineRadio.OnlineRadioStation.models.Artist;
import com.OnlineRadio.OnlineRadioStation.models.FullSong;
import com.OnlineRadio.OnlineRadioStation.models.Song;

import java.util.List;
import java.util.stream.Collectors;

public class FullSongToSongAdapter implements SongAdapter {
    private FullSong fullSong;

    public FullSongToSongAdapter(FullSong fullSong) {
        this.fullSong = fullSong;
    }

    @Override
    public Song adapt() {
        List<String> artistIds = fullSong.getArtists()
            .stream()
            .map(Artist::getId)
```

```

        .collect(Collectors.toList());

        return new Song(fullSong.getId(), fullSong.getTitle(), artistIds);
    }
}

```

Рисунок 2. Інтерфейс SongAdapter

Використання у SongService адаптора FullSongToSongAdapter:

```

package com.OnlineRadio.OnlineRadioStation.services;

import com.OnlineRadio.OnlineRadioStation.adapters.FullSongToSongAdapter;
import com.OnlineRadio.OnlineRadioStation.models.FullSong;
import com.OnlineRadio.OnlineRadioStation.models.Song;
import com.OnlineRadio.OnlineRadioStation.iterators.SongIterator;
import com.OnlineRadio.OnlineRadioStation.iterators.RandomSongIterator;

import java.util.ArrayList;
import java.util.List;

public class SongService {
    private List<Song> songs;

    public SongService() {
        this.songs = new ArrayList<>();
    }

    public void addSong(Song song) {
        songs.add(song);
    }

    public void addSong(FullSong fullSong) {
        FullSongToSongAdapter adapter = new
FullSongToSongAdapter(fullSong);
        songs.add(adapter.adapt());
    }

    public List<Song> getSongs() {
        return songs;
    }

    public SongIterator getSongIterator() {
        return new SongIterator(songs);
    }

    public RandomSongIterator getRandomSongIterator() {
        return new RandomSongIterator(songs);
    }
}

```

Рисунок 3. Приклад використання у SongService адаптора FullSongToSongAdapter

Тестування адаптору

```

package com.OnlineRadio.OnlineRadioStation.adapters;

```

```

import com.OnlineRadio.OnlineRadioStation.models.Artist;
import com.OnlineRadio.OnlineRadioStation.models.FullSong;
import com.OnlineRadio.OnlineRadioStation.models.Song;
import org.junit.jupiter.api.Test;

import java.util.List;

import static org.junit.jupiter.api.Assertions.assertEquals;

class FullSongToSongAdapterTest {
    @Test

    void testFullSongToSongAdapter() {
        Artist artist1 = new Artist("1", "Artist 1");
        Artist artist2 = new Artist("2", "Artist 2");
        FullSong fullSong = new FullSong("123", "Test Song",
List.of(artist1, artist2));

        FullSongToSongAdapter adapter = new
FullSongToSongAdapter(fullSong);
        Song adaptedSong = adapter.adapt();

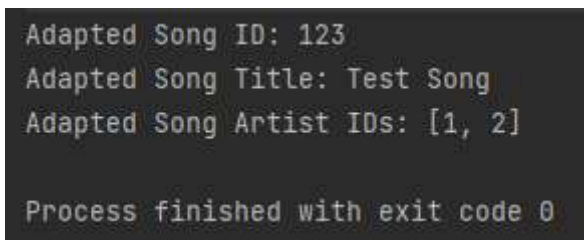
        System.out.println("Adapted Song ID: " + adaptedSong.getId());
        System.out.println("Adapted Song Title: " +
adaptedSong.getTitle());
        System.out.println("Adapted Song Artist IDs: " +
adaptedSong.getArtistIds());

        assertEquals("123", adaptedSong.getId());
        assertEquals("Test Song", adaptedSong.getTitle());
        assertEquals(List.of("1", "2"), adaptedSong.getArtistIds());
    }
}

```

Рисунок 4. Тестування адаптору

Результати тестування



```

Adapted Song ID: 123
Adapted Song Title: Test Song
Adapted Song Artist IDs: [1, 2]

Process finished with exit code 0

```

Рисунок 5. Тестування адаптору

Висновок: На цій лаб. роботі я зрозумів суть шаблонів шаблони «ADAPTER», «BUILDER», «COMMAND», «CHAIN OF RESPONSIBILITY», «PROTOTYPE», реалізував один з них (Adaptor) для свого проекту “Online radio station”, оскільки у

проекті може виникнути потребу у адаптації інтерфейсу одного об'єкту до іншого, наприклад Song з списком виконавців у Song зі списком їх Id, з чим і допомагає адаптер.